

多智能体协作开发《AI 社交成长教练》傻瓜教程 v0.1

0. 你现在手里的工具情况

你现在大概有这些资源：

1. Kimi Code 接入 Claude Code

适合：复杂代码生成、架构改造、长上下文理解、代码审查、重构。

2. Kimi Code / KimiCode 额度充足

适合：作为主力开发智能体，承担最多的编码工作。

3. 接入 DeepSeek 的 Hermes

适合：低成本推理、后端逻辑、代码解释、测试用例、批量生成 prompt、SQL、配置文件。

4. Cursor 额度不确定

适合：本地代码编辑、快速补全、小范围改代码、文件间跳转、人工驾驶开发。

5. Gemini 免费版

适合：辅助查资料、对比方案、生成 UI 文案、做第二意见，不作为主力编码。

6. DeepSeek API 余额约 5 元

适合：仅用于真实接口联调、低成本测试、备用模型验证，不要拿来大规模开发。

7. KimiCode 额度充足

结论：它应该是你的主力施工工具。

1. 总体策略：一个老板，几个员工，不要让智能体互相乱改

你要把这些 AI 当成一个小团队，而不是同时打开一堆窗口让它们乱写。

最推荐的协作模式是：

你 = 产品经理 + 最终拍板人
Kimi Code = 主力工程师
Claude Code = 架构师 + 高级代码审查
Hermes / DeepSeek = 测试工程师 + 便宜劳动力
Cursor = 你的 IDE 和小修小补工具
Gemini = 搜索助理 + 文案助理 + 第二意见

一句话：

Kimi Code 负责写，Claude Code 负责审，Cursor 负责改，Hermes 负责测，Gemini 负责查。

2. 最重要的纪律：每次只让一个智能体改代码

这是成败关键。

不要这样做：

Claude 在改后端
Kimi 在改前端
Cursor 也在自动修 bug
Hermes 又生成了一套新目录

这样很快会变成灾难。

正确方式：

第 1 步：让 Claude / Kimi 出方案
第 2 步：你确认采用哪套方案
第 3 步：只让 Kimi Code 改代码
第 4 步：跑测试
第 5 步：把错误交给 Kimi 修
第 6 步：最后让 Claude Code 审查

规则：

1. 同一时间只有一个 AI 有“写代码权”。
 2. 其他 AI 只做“审查、建议、解释、生成测试”。
 3. 每完成一个小功能就 git commit。
 4. 不要一次让 AI 做超过 1 个大模块。
 5. 所有 AI 修改前，先让它读 README / docs / 当前目录结构。
-

3. 推荐项目技术栈

为了降低难度，建议第一版这样做：

前端：Next.js + React + Tailwind CSS
后端：FastAPI
Agent：LangGraph
模型调用：先封装成统一 LLM Provider
数据库：SQLite 起步，后面换 PostgreSQL
语音：先做浏览器录音 + STT/TTS 接口占位

向量库：第一版先不做，后面加 pgvector
部署：先本地跑通，再 Docker，再云部署

为什么先用 SQLite？

因为 MVP 阶段你最需要的是跑通产品闭环，不是搭复杂基础设施。

MVP 闭环是：

```
graph TD; A[用户选择训练场景] --> B[用户输入文字/语音]; B --> C[Agent 分析]; C --> D[生成表达建议]; D --> E[角色扮演继续追问]; E --> F[训练结束后评分]; F --> G[保存训练记录];
```

4. 建议先做的 MVP 功能

不要一开始做完整产品。先做这 6 个功能：

第一阶段：文字版 MVP

1. 首页：显示训练入口。
2. 场景选择：校园 / 职场 / 面试 / 朋友 / 观点表达。
3. 文字对话训练：用户输入一句话，AI 给反馈。
4. 观点表达训练：用 PREP 结构给用户改写。
5. 角色扮演：AI 扮演同学/领导/面试官。
6. 训练复盘：给清晰度、逻辑性、自信度、礼貌度评分。

第二阶段：语音 MVP

1. 前端录音。
2. 上传音频。
3. 转写为文字。
4. 调用 Agent。
5. TTS 播放回复。
6. 给语音反馈：语速、停顿、犹豫词。

第三阶段：成长系统

1. 用户训练记录。
2. 能力评分趋势。
3. 7 天训练计划。
4. 社交礼仪课程。

5. 项目目录建议

让 Kimi Code 按这个目录创建项目：

```
social-coach-ai/  
├── README.md  
├── docs/  
│   ├── PRD.md  
│   ├── TECH_DESIGN.md  
│   ├── PROMPTS.md  
│   └── TASKS.md  
├── apps/  
│   └── web/  
│       ├── app/  
│       ├── components/  
│       ├── lib/  
│       └── package.json  
├── services/  
│   └── api/  
│       ├── app/  
│       │   ├── main.py  
│       │   ├── routers/  
│       │   ├── graph/  
│       │   ├── services/  
│       │   ├── models/  
│       │   └── schemas/  
│       ├── tests/  
│       ├── requirements.txt  
│       └── .env.example  
├── prompts/  
│   ├── system_social_coach.md  
│   ├── expression_coach.md  
│   ├── roleplay_agent.md  
│   ├── etiquette_coach.md  
│   └── safety_guard.md  
└── scripts/  
    ├── dev.sh  
    └── test.sh
```

6. 各智能体怎么分工

6.1 Kimi Code：主力工程师

用它做什么

- 创建项目目录。
- 写前端页面。
- 写 FastAPI 接口。
- 写 LangGraph 节点。
- 写数据库模型。
- 修 bug。
- 根据错误日志改代码。
- 生成 README。

不建议让它做什么

- 一次性生成整个完整应用。
- 同时改前端、后端、数据库、语音、部署。
- 在你没确认方案前大规模重构。

给 Kimi Code 的标准指令模板

你是本项目的主力工程师。请先阅读 README.md、docs/PRD.md、docs/TECH_DESIGN.md 和当前目录结构。

本次任务只做一件事：{任务名称}。

要求：

1. 不要改动与本任务无关的文件。
2. 不要大规模重构。
3. 如果需要新增依赖，先说明原因。
4. 代码要能本地运行。
5. 完成后告诉我：修改了哪些文件、如何启动、如何测试。

现在任务是：{具体任务}

6.2 Claude Code：架构师 + 代码审查

用它做什么

- 审查 Kimi 写的代码。
- 检查架构是否合理。
- 发现安全隐患。
- 优化 LangGraph 状态设计。
- 优化 prompts。
- 检查是否过度工程化。

不建议让它做什么

- 在没有计划时直接重写项目。
- 和 Kimi 同时修改同一批文件。

给 Claude Code 的审查模板

你现在是本项目的架构师和代码审查员。请审查当前代码，不要直接修改。

重点检查：

1. 是否符合 docs/TECH_DESIGN.md 的架构。
2. 前后端接口是否清晰。
3. LangGraph 状态设计是否可维护。
4. 是否存在明显 bug。
5. 是否存在安全风险，尤其是心理危机、未成年人、操控话术相关逻辑。
6. 是否有过度工程化。

请输出：

- 必须修复的问题
- 建议优化的问题
- 可以暂时不管的问题
- 建议交给 Kimi Code 执行的具体任务清单

6.3 Hermes / DeepSeek：测试工程师 + 便宜劳动力

用它做什么

- 生成测试用例。
- 写单元测试。
- 生成 mock 数据。
- 写 SQL。
- 检查 prompt 是否稳定。
- 生成安全测试样本。
- 解释报错。

不建议让它做什么

- 做核心架构决策。
- 大规模改代码。
- 接手复杂前后端联调。

给 Hermes 的测试模板

你是本项目的测试工程师。请根据下面的功能描述生成测试用例，不要改生产代码。

功能：{功能描述}

请输出：

1. 正常用例。
2. 边界用例。
3. 异常用例。
4. 安全风险用例。
5. 如果适合，请生成 pytest 测试代码。

项目背景：这是一个 AI 社交成长教练，必须避免心理诊断、操控话术、PUA、未成年人风险和自伤危机场景误处理。

6.4 Cursor：本地驾驶舱

用它做什么

- 打开项目。
- 查看文件结构。
- 小范围修改。
- 自动补全。
- 根据终端错误快速定位。
- 人工审查 AI 改动。

不建议让它做什么

- 额度不确定时，不要让 Cursor Composer 大规模生成项目。
- 不要让它和 Kimi 同时重构。

Cursor 使用规则

1. 大功能交给 Kimi Code。
2. 小改动用 Cursor。
3. 报错定位用 Cursor。
4. 最后人工检查 diff。

6.5 Gemini 免费版：资料员 + 文案员

用它做什么

- 查找某个技术方案优缺点。
- 生成 UI 文案。
- 给社交礼仪课程生成提纲。
- 对 PRD 或技术文档给第二意见。
- 生成 onboarding 问题。

不建议让它做什么

- 主力写代码。
- 处理你的私有代码仓库。
- 做涉及密钥、用户数据、真实隐私的内容。

给 Gemini 的模板

我正在做一个 AI 社交成长教练 App，目标用户是低社交度的青少年、大学生和初入职场人群。

请帮我生成：{内容}

要求：

1. 语言适合年轻用户。
2. 不要像心理治疗。
3. 不要制造焦虑。
4. 强调练习、成长、自信表达和社交礼仪。

6.6 DeepSeek API 余额 5 元：省着用

用它做什么

- 测试你自己的 LLM Provider 封装。
- 低成本跑少量真实接口。
- 作为备用模型。
- 测试 prompt 在 DeepSeek 上的效果。

不要做什么

- 不要让它承担大规模对话。
- 不要拿它批量生成内容。
- 不要做实时语音主链路。

7. 从零开始手把手开发流程

第 1 步：创建 Git 仓库

在终端执行：

```
mkdir social-coach-ai
cd social-coach-ai
git init
```

创建基础文件：

```
mkdir docs prompts apps services scripts
touch README.md docs/PRD.md docs/TECH_DESIGN.md docs/TASKS.md
```

把我们前面生成的 PRD 和技术设计文档复制进去：


```
PRD → docs/PRD.md  
技术设计文档 → docs/TECH_DESIGN.md
```

提交：

```
git add .  
git commit -m "docs: add initial PRD and technical design"
```

第 2 步：让 Claude Code 审一遍技术方案

给 Claude Code：

你是本项目架构师。请阅读 docs/PRD.md 和 docs/TECH_DESIGN.md。

请检查这个 MVP 技术方案是否过度复杂，并输出一个最小可落地版本。

要求：

1. 第一版必须能在本地跑通。
2. 不要引入太多服务。
3. 优先保证“文字训练 + 观点表达 + 角色扮演 + 复盘评分”。
4. 语音功能先保留接口，不一定第一天实现。
5. 输出具体开发任务清单。

你得到任务清单后，把清单整理进：

```
docs/TASKS.md
```

提交：

```
git add docs/TASKS.md  
git commit -m "docs: add MVP task list"
```

第 3 步：让 Kimi Code 初始化前端

给 Kimi Code：

你是本项目主力工程师。请先阅读 README.md、docs/PRD.md、docs/TECH_DESIGN.md、docs/TASKS.md。

本次任务只初始化前端项目，不要写后端。

要求：

1. 在 apps/web 下创建 Next.js + TypeScript + Tailwind 项目。
2. 实现 4 个静态页面：
 - 首页
 - 场景训练页
 - 表达训练页
 - 成长复盘页
3. 先使用 mock 数据，不连接后端。
4. UI 风格参考：年轻、干净、圆角、紫色主色、移动端优先。
5. 完成后告诉我启动命令。

然后你本地运行：

```
cd apps/web
npm install
npm run dev
```

检查没问题后：

```
git add .
git commit -m "feat(web): initialize mobile-first MVP UI"
```

第 4 步：让 Kimi Code 初始化后端

给 Kimi Code：

你是本项目主力工程师。请阅读 docs/TECH_DESIGN.md 和当前目录结构。

本次任务只初始化 FastAPI 后端，不要改前端。

要求：

1. 在 services/api 下创建 FastAPI 项目。
2. 提供健康检查接口 GET /health。
3. 提供 POST /agent/turn 接口。
4. 暂时使用 mock agent 返回固定结构。
5. 加入 Pydantic 请求/响应 schema。
6. 提供 requirements.txt 和启动说明。

运行：

```
cd services/api
python -m venv .venv
source .venv/bin/activate
```

```
pip install -r requirements.txt
uvicorn app.main:app --reload
```

测试:

```
curl http://localhost:8000/health
```

提交:

```
git add .
git commit -m "feat(api): initialize FastAPI service with mock agent endpoint"
```

第 5 步：前后端联调

给 Kimi Code:

本次任务只做前后端联调。

要求:

1. 前端表达训练页调用后端 POST /agent/turn。
2. 后端返回 mock 反馈。
3. 前端展示: AI 回复、评分、改写版本、下一步练习。
4. 不要引入真实 LLM。
5. 保持页面移动端体验。

你运行两个服务:

```
# 终端 1
cd services/api
source .venv/bin/activate
uvicorn app.main:app --reload

# 终端 2
cd apps/web
npm run dev
```

测试浏览器能不能正常调用。

提交:

```
git add .
git commit -m "feat: connect web expression training to mock agent API"
```

第 6 步：实现 LLM Provider 封装

目标：以后你可以切换 Kimi、DeepSeek、Claude、OpenAI 等模型。

给 Kimi Code：

本次任务只实现后端 LLM Provider 封装，不改前端页面。

要求：

1. 在 `services/api/app/services/llm` 下实现统一接口。
2. 支持环境变量选择 `provider: mock、deepseek、kimi`。
3. 默认 `provider=mock`。
4. 不要把 API Key 写死在代码里。
5. 提供 `.env.example`。
6. `/agent/turn` 继续可用。
7. 如果 `provider=mock`，不需要真实调用模型。

建议环境变量：

```
LLM_PROVIDER=mock
DEEPSEEK_API_KEY=
KIMI_API_KEY=
MODEL_NAME=
```

提交：

```
git add .
git commit -m "feat(api): add pluggable LLM provider abstraction"
```

第 7 步：让 Hermes 生成测试用例

给 Hermes：

你是测试工程师。请为 `POST /agent/turn` 生成 `pytest` 测试。

接口功能：

用户输入一段社交/表达训练文本，后端返回 AI 回复、评分、改写版本、下一步练习。

要求测试：

1. 正常表达训练输入。
2. 空输入。
3. 超长输入。
4. 自伤风险输入。

5. 操控/PUA 请求。
6. 未成年人风险场景。

请生成 pytest 代码和测试数据。

把测试代码交给 Kimi Code 集成。

给 Kimi Code：

请把下面的 pytest 测试集成到 services/api/tests 中。

要求：

1. 不要改业务逻辑，除非测试发现明显问题。
2. 确保 pytest 能运行。
3. 如果当前 mock agent 无法通过安全测试，请先用最小逻辑补齐 safety guard。

运行：

```
cd services/api
pytest
```

提交：

```
git add .
git commit -m "test(api): add agent turn safety and normal case tests"
```

第 8 步：实现 LangGraph 最小图

给 Kimi Code：

本次任务实现 LangGraph 最小可用图，不要改前端 UI。

目标：替换 /agent/turn 里的 mock agent，但保留 mock LLM provider 可用。

图节点：

1. safety_node：判断是否为普通训练、危机、操控请求。
2. intent_node：判断是场景分析、表达训练、角色扮演、礼仪学习。
3. coach_node：生成训练反馈。
4. feedback_node：生成评分和下一步练习。

要求：

1. 使用 TypedDict 定义 AgentState。
2. 输出结构必须和现有前端兼容。
3. provider=mock 时仍然不用真实模型。

4. provider=kimi 或 deepseek 时可以调用真实模型。
5. 添加最小测试。

跑测试：

```
pytest
```

提交：

```
git add .  
git commit -m "feat(agent): implement minimal LangGraph coaching workflow"
```

第 9 步：让 Claude Code 审查 LangGraph 实现

给 Claude Code：

请审查当前 LangGraph 实现，不要直接改代码。

重点检查：

1. AgentState 是否清晰。
2. safety_node 是否在普通训练前执行。
3. 输出结构是否稳定。
4. 是否方便以后增加语音、记忆、角色扮演。
5. 是否有过度工程化。
6. 是否有明显 bug。

请输出：

- 必须修复
- 建议修复
- 暂时不用管
- 给 Kimi Code 的具体修改任务

再把 Claude 的“必须修复”交给 Kimi Code。

提交：

```
git add .  
git commit -m "fix(agent): address architecture review feedback"
```

第 10 步：接入真实 Kimi 模型

因为你的 KimiCode 额度充足，真实模型优先用 Kimi。

给 Kimi Code:

本次任务只接入 Kimi Provider。

要求:

1. 使用环境变量 KIMI_API_KEY、KIMI_MODEL。
2. 不要写死 key。
3. provider=mock 时不受影响。
4. provider=kimi 时 /agent/turn 使用 Kimi 生成结果。
5. 保证输出 JSON 可以解析。如果模型输出不合法,要有 fallback。
6. 加入超时和错误处理。

.env 示例:

```
LLM_PROVIDER=kimi  
KIMI_API_KEY=你的key  
KIMI_MODEL=你的模型名
```

测试:

```
curl -X POST http://localhost:8000/agent/turn  
-H "Content-Type: application/json"  
-d '{"user_id":"demo","session_id":"s1","mode":"expression_training","text":"我觉得这个方案可能不太好,但我不知道怎么说"}'
```

提交:

```
git add .  
git commit -m "feat(api): add Kimi LLM provider"
```

第 11 步: DeepSeek API 只做备用 provider

给 Kimi Code:

本次任务只添加 DeepSeek Provider 作为备用。

要求:

1. 使用 DEEPSEEK_API_KEY、DEEPSEEK_MODEL。
2. provider=deepseek 时调用 DeepSeek。
3. 不要影响 mock 和 kimi provider。
4. 加入错误处理。
5. 注意余额有限,不要写会大量调用 API 的测试。

测试一次即可，不要疯狂跑。

提交：

```
git add .
git commit -m "feat(api): add DeepSeek fallback provider"
```

第 12 步：实现数据库保存训练记录

MVP 用 SQLite。

给 Kimi Code：

本次任务只实现训练记录保存。

要求：

1. 使用 SQLite + SQLAlchemy。
2. 新增 training_sessions 和 training_turns 两张表。
3. /agent/turn 每次调用后保存用户输入和 AI 输出。
4. 新增 GET /training/sessions/{session_id} 查看历史。
5. 不要引入 PostgreSQL。
6. 添加最小测试。

提交：

```
git add .
git commit -m "feat(api): persist training sessions and turns in SQLite"
```

第 13 步：做语音 MVP 的空壳

先不要急着接真实 STT/TTS。

给 Kimi Code：

本次任务实现语音模块接口空壳。

要求：

1. 新增 POST /voice/transcribe。
2. 接收音频文件，但 MVP 阶段可以返回 mock transcript。
3. 新增 POST /voice/synthesize。
4. 输入 text，返回 mock audio_url 或 base64 占位。

5. 前端语音陪练页面可以录音并调用接口。
6. 不接真实第三方语音服务。

提交：

```
git add .  
git commit -m "feat(voice): add mock voice transcribe and synthesize endpoints"
```

第 14 步：接真实 STT/TTS

等文字版稳定后再做。

给 Kimi Code：

本次任务接入真实 STT/TTS。

要求：

1. 保留 mock voice provider。
2. 新增真实 voice provider。
3. 使用环境变量 VOICE_PROVIDER。
4. /voice/transcribe 返回 transcript 和基础 audio_features。
5. /voice/synthesize 返回可播放音频。
6. 不保存原始音频，除非配置 SAVE_AUDIO=true。
7. 添加错误处理和大小限制。

提交：

```
git add .  
git commit -m "feat(voice): add real STT and TTS provider support"
```

8. 每天开发节奏

你每天按这个节奏来，最稳：

```
上午：确定今天只做 1-2 个任务  
↓  
让 Kimi Code 实现  
↓  
本地运行  
↓  
把错误日志给 Kimi 修  
↓
```

```
提交 git
↓
让 Claude Code 审查
↓
把必须修复项给 Kimi
↓
再提交
```

每天不要做超过 3 个大任务。

每个任务都要满足：

```
能启动
能访问
能测试
能回滚
```

9. Git 分支策略

如果你是一个人开发，简单点：

```
main: 稳定版本
feature/ui-mvp: 前端 MVP
feature/api-mvp: 后端 MVP
feature/langgraph: Agent 图
feature/voice-mvp: 语音 MVP
```

每个功能做完：

```
git checkout main
git merge feature/api-mvp
git tag v0.1-api-mvp
```

如果你嫌分支麻烦，至少每完成一个小任务 commit 一次。

10. 任务拆分清单

阶段 A：项目骨架

- [] 创建仓库。
- [] 写入 PRD。
- [] 写入技术设计文档。

- ☐ 写 TASKS.md。
- ☐ 初始化前端。
- ☐ 初始化后端。
- ☐ 前后端 mock 联调。

阶段 B：核心 Agent

- ☐ LLM Provider 抽象。
- ☐ Mock provider。
- ☐ Kimi provider。
- ☐ DeepSeek provider。
- ☐ LangGraph AgentState。
- ☐ Safety Node。
- ☐ Intent Node。
- ☐ Coach Node。
- ☐ Feedback Node。
- ☐ 结构化输出解析。

阶段 C：核心功能

- ☐ 场景训练。
- ☐ 表达训练。
- ☐ 观点表达 PREP 训练。
- ☐ 角色扮演。
- ☐ 社交礼仪列表。
- ☐ 训练复盘。

阶段 D：数据保存

- ☐ 用户画像。
- ☐ 训练会话。
- ☐ 对话轮次。
- ☐ 能力评分。
- ☐ 成长趋势。

阶段 E：语音 MVP

- ☐ 前端录音。
- ☐ 上传音频。
- ☐ STT 接口。
- ☐ TTS 接口。
- ☐ 语音陪练页。
- ☐ 语音反馈指标。

阶段 F：安全与测试

- ☐ 自伤风险测试。
- ☐ 伤人风险测试。
- ☐ PUA/操控请求测试。
- ☐ 未成年人风险测试。

- [] 普通训练回归测试。
 - [] Prompt 输出格式测试。
-

11. 你该怎么用额度

KimiCode 额度充足

用在：

- 大量编码。
- 修 bug。
- 写前端。
- 写后端。
- 写 LangGraph。
- 写 README。
- 生成代码注释。

Claude Code

用在：

- 每个阶段结束后的审查。
- 复杂 bug。
- 架构重构建议。
- prompt 质量提升。
- 安全边界审查。

Cursor

用在：

- 小范围修改。
- 快速定位。
- 看 diff。
- 人工驾驶。

Hermes / DeepSeek

用在：

- 生成测试。
- 解释错误。
- 批量 mock 数据。
- SQL。
- 安全用例。

Gemini 免费版

用在：

- UI 文案。
- 社交礼仪内容。
- 用户引导话术。
- 功能命名。
- 查资料。

DeepSeek API 余额 5 元

只用在：

- provider 是否跑通。
- 少量真实调用测试。
- 备用模型效果对比。

12. 最推荐的执行顺序

按这个顺序做，不要跳：

1. 文档入仓
2. 前端静态原型
3. 后端 mock API
4. 前后端联调
5. LLM provider 抽象
6. LangGraph 最小图
7. Kimi provider 接入
8. 训练记录保存
9. 表达训练完善
10. 角色扮演完善
11. 社交礼仪模块
12. 语音接口 mock
13. 真实 STT/TTS
14. 安全测试集
15. 部署

千万不要一上来就做：

实时语音
多人训练房间
复杂 RAG
支付系统

这些都放后面。

13. 每个阶段的验收标准

前端静态原型验收

- [] 手机尺寸显示正常。
- [] 首页有训练入口。
- [] 表达训练页能输入文字。
- [] 复盘页能展示评分。
- [] 页面风格统一。

后端 mock API 验收

- [] /health 正常。
- [] /agent/turn 正常。
- [] 返回结构稳定。
- [] 前端可以显示返回数据。

LangGraph 验收

- [] safety_node 总是最先执行。
- [] 普通输入能进入训练。
- [] 危机输入能进入安全响应。
- [] 操控请求会被拒绝并转为健康表达。
- [] 输出 JSON 稳定。

Kimi Provider 验收

- [] API Key 不写死。
- [] provider=mock 可用。
- [] provider=kimi 可用。
- [] 模型输出不合法时有 fallback。
- [] 超时不导致服务崩溃。

语音 MVP 验收

- [] 前端能录音。
- [] 后端能接收音频。
- [] 能生成 transcript。
- [] 能返回 AI 回复。
- [] 能播放 TTS。

14. 常见坑

坑 1：让 AI 一次做太多

错误：

帮我完整实现这个项目，包括前后端、数据库、语音、部署。

正确：

本次只实现 POST /agent/turn，使用 mock 返回，不要改前端。

坑 2：没有 mock 层

没有 mock，你会很快被模型额度、API 错误、网络问题拖死。

一定要支持：

```
LLM_PROVIDER=mock  
VOICE_PROVIDER=mock
```

坑 3：没有稳定输出格式

Agent 必须返回固定 JSON。否则前端会一直崩。

要求模型输出：

```
{  
  "reply_text": "...",  
  "scores": {...},  
  "rewritten_expression": "...",  
  "next_practice": "...",  
  "safety": {...}  
}
```

坑 4：过早做实时语音

实时语音很诱人，但第一版会吞掉大量时间。

先做：

按住说话 → 转文字 → Agent → TTS

再做：

WebRTC 实时语音

坑 5：安全模块后补

安全模块必须从第一天就有，哪怕很简单。

最少要识别：

自伤
伤人
PUA/操控
未成年人风险
心理诊断请求

15. 可以直接复制给 Kimi Code 的总启动 Prompt

你是本项目的主力全栈工程师。我们要开发一个 AI 社交成长教练 App，目标用户是低社交度的青少年、大学生和初入职场人群。

产品目标：

帮助用户通过场景训练、观点表达训练、角色扮演、语音陪练、社交礼仪学习和复盘反馈，提升社交自信、逻辑表达、沟通能力和边界感。

技术路线：

- 前端：Next.js + React + TypeScript + Tailwind
- 后端：FastAPI
- Agent：LangGraph
- LLM：先 mock，后续支持 Kimi 和 DeepSeek
- 数据库：MVP 用 SQLite
- 语音：先 mock STT/TTS，后续接真实服务

开发原则：

1. 每次只做一个小任务。
2. 不要大规模重构。
3. 不要改无关文件。
4. 所有模型调用都要有 mock provider。

5. API Key 只能通过环境变量读取。
6. 后端输出必须结构化 JSON。
7. safety_node 必须在普通训练前执行。
8. 不提供心理诊断、PUA、操控、骚扰、自伤鼓励等内容。
9. 完成后告诉我修改了哪些文件、如何启动、如何测试。

请先阅读 README.md、docs/PRD.md、docs/TECH_DESIGN.md、docs/TASKS.md 和当前目录结构，然后等待我给你具体任务。不要现在就开始改代码。

16. 可以直接复制给 Claude Code 的审查 Prompt

你是本项目的架构师、代码审查员和安全审查员。请阅读 docs/PRD.md、docs/TECH_DESIGN.md、docs/TASKS.md 和当前代码。

项目是 AI 社交成长教练，目标用户是低社交度的青少年、大学生和初入职场人群。核心功能包括社交训练、观点表达训练、角色扮演、语音陪练、社交礼仪和复盘反馈。

请不要直接改代码，只做审查。

重点检查：

1. 架构是否适合 MVP。
2. 是否过度工程化。
3. 前后端接口是否稳定。
4. LangGraph 状态和节点是否清晰。
5. safety_node 是否在普通训练前执行。
6. 输出 JSON 是否稳定。
7. 是否存在 API Key 泄露风险。
8. 是否正确处理自伤、伤人、未成年人、PUA/操控请求。
9. 是否方便后续加入语音和成长记录。

请输出：

- 必须修复的问题
- 建议优化的问题
- 可以暂时不管的问题
- 建议交给 Kimi Code 的具体任务清单

17. 可以直接复制给 Hermes / DeepSeek 的测试 Prompt

你是本项目的测试工程师。请根据下面的功能生成测试用例。

项目背景：AI 社交成长教练，目标用户是低社交度青少年、大学生和初入职场人群。功能包括社交训练、观点表达训练、角色扮演、语音陪练、社交礼仪和复盘反馈。

安全要求：

- 不做心理诊断。
- 不提供 PUA、操控、骚扰话术。
- 遇到自伤/伤人风险要安全分流。
- 未成年人风险要更谨慎。

功能：{填入具体功能}

请输出：

1. 正常测试用例。
2. 边界测试用例。
3. 异常测试用例。
4. 安全测试用例。
5. pytest 示例代码。

18. 最后给你的个人操作建议

你现在最应该做的是：

第一天：文档入仓 + 前端静态页面
第二天：后端 mock API + 前后端联调
第三天：LLM Provider + LangGraph 最小图
第四天：Kimi Provider + 安全节点
第五天：表达训练 + 角色扮演
第六天：训练记录 + 复盘页
第七天：语音 mock + 整体演示

七天内不要追求完整产品，只追求一个可以演示的闭环：

打开首页
↓
选择观点表达训练
↓
输入一句不自信的话
↓
AI 给出结构化反馈
↓
用户重新练习
↓
系统评分并保存记录

只要这个闭环跑通，你的项目就已经有雏形了。后面的语音、礼仪课程、实时陪练、成长计划都是在这个闭环上加东西。

19. 最简版指挥口诀

你可以记这个：

Kimi 写代码，Claude 看架构，Hermes 写测试，Cursor 小修补，Gemini 查资料，DeepSeek 省着用。

更重要的是：

一次只做一件事。
每步都能运行。
每步都要提交。
模型都要可替换。
安全从第一天就做。